Project Assignment 1: Use Case Definition, Software Architecture Design, and Pattern Mapping

This project is structured into three assignments, in parallel with the project implementation and the final project documentation. Each document you submit throughout the term must include a Table of Contents at the beginning, listing all the required sections.

Assignments and Deliverables (Project Output Packages)

For each assignment, you are required to create a project output package and upload it to GitHub. Think of each assignment as a step in planning and defining your project. The "project output package" for each assignment will consist of specific documents.

Important Notes about GitHub and Submissions

GitHub Submission: You will submit your work for each assignment through GitHub. This means you need to upload your project output package (the documents for each assignment) to your GitHub repository.

Individual GitHub Accounts: Each team member should have their own GitHub account. This is important so we can clearly track individual contributions and who is responsible for what part of the project.

Clarity of Contribution: On GitHub, it should be evident which team member is working on which aspects of the project. Use clear commit messages and file structures to show individual contributions.

Document Authorship: On the first page of each document (Project Definition Document, Project Plan Document, Project Requirements Document), please list the names of the team members who contributed to the creation of that specific document.

Repository Privacy Options

Public Repository (Default): You are welcome to use a public GitHub repository for your project.

Private Repository (Optional): You also have the option to create a private repository on GitHub if you prefer to keep your project code and documentation private and only visible to your team members. If you choose a private repository, you must grant me access to your repository.

Task Matrix in Each Document

Document-Specific Task Matrix: Within each document you submit (Project Definition, Project Plan, Requirements), you are required to include a document-specific task matrix. This matrix should detail the tasks involved in creating that specific document and clearly show which team member was responsible for each task within the document's creation process. This helps illustrate the division of work within your team for each deliverable

Deadline: 15 June 2025, 23:59

Submission Instructions:

• Each team is required to submit their work on GitHub, ensuring clear commit messages and contribution tracking for each member.

• Maintain the traceability via numbering and naming conventions throughout the design, implementation and test for the four selected use case.

Deliverables:

- Project Definition Document, Project Plan Document, Software Requirements Specification Document
- Software Architecture Overview (diagram and explanation)
- Pattern-Use Case Mapping matrix with explanation and justifications (minimum one pattern per use case)

Objective:

To develop a foundational understanding of your project's functionality by documenting key use cases, proposing a software architecture, and demonstrating how design patterns can be applied to support each use case effectively.

Instructions:

1. **Prepare**
    a. **Project Definition Document**
    b. **Project Plan Document**
2. **Four Core Use Cases in a Software Requirements Specification Document**
    a. Identify and describe **three primary use cases** for your project.

b. For each use case, include:
    i. **Title**
    ii. **Main Actor**
    iii. **Goal**
    iv. **Preconditions**
    v. **Main Flow** (sequence of steps)
    vi. **Postconditions**

3. **Design Software Architecture**
    a. Provide a **high-level architecture design** that supports the identified use cases.
    b. Include a **diagram or structured description** that shows major architectural components (e.g., client-server, layered architecture, microservices, etc.).
    c. Briefly explain how the architecture facilitates the realization of the use cases.
4. **Apply Design Patterns**
    a. For **each use case**, identify and describe **at least one relevant software design pattern**.
        i. Name the pattern
        ii. Explain **why** it is appropriate for the use case
        iii. Describe **how** it will be applied in the architecture to enhance flexibility, modularity, or maintainability

1. Project Definition Document

- Project Name: Give the project a meaningful name.
- Project Summary (1-2 paragraphs): Briefly and concisely explain what the project is,what problem it solves, and its main purpose.
- Objectives: What exactly do you want to achieve by the end of this project? Think aboutthe functionalities you aim to implement.
- Scope: Clearly define the boundaries of your project - what is included and, importantly,what is NOT included.
- Target Audience: Specify who the project is designed for (e.g., students, book lovers,event organizers, etc.).
- Key Features (At least 5 items): List the core functionalities of the project and the mainfeatures it will offer to users. For example, for a "To-Do List Application": Add task, deletetask, complete task, prioritize, categorize.
- Deliverables

- Project Success Criteria (At least 3 items): Define criteria to measure whether the projectis successful. For example, "80% of users will find the application easy to use", "The application will fully implement all 5 key features."

2. Project Plan Document:

- Project Scope: Clearly define what the project will include and what it will not include. This is important for setting project boundaries.
- Project Organization - People (roles and responsibilities)
- Objectives: Define objectives specifically for the project execution and management.
- Key Phases and Timeline (Simple Gantt Chart or List): Create a simple timeline showing the essential phases required to complete the project (e.g., requirements gathering, design, development, testing, deployment) and the estimated start and end dates for each phase. This illustrates the project duration and when phases should be completed. Delivery dates and deliverables are explained and mapped to the timeline. Closure date and project evaluation date.
- Resource Planning (Simple): Specify the basic resources required for the project (e.g., software tools, libraries, database, etc.) and how these resources will be accessed.
- Risk Management (Simple): Briefly describe potential risks that may be encountered during the project (e.g., technical difficulties, delays, team member leaving, etc.) and the measures that can be taken to mitigate these risks.
- Communication Plan (Simple): Briefly describe how communication will be ensured among team members (e.g., weekly meetings, online communication tools, etc.) and how communication with project stakeholders (if any) will be conducted.
- Change Management Plan: Outline a simple process for managing changes to project requirements or scope.-Budget Plan: Explain how project effort and resources will be allocated across different phases and tasks.
- Acceptance Tests and Acceptance Criteria

3. Software Requirements Document

- Product Perspective: Describe where this software fits in (e.g., a stand-alone app, part of a larger course project).
- Product Functions: Summarize main functions the system will perform.
- User Characteristics: Describe the expected users and their skill levels.
- Constraints: Mention any platform, language, or deadline limitations.
- System Features (Use Case Based): This section breaks down the system based on user interactions.

- Use Case Diagram: Insert or describe a UML use case diagram representing system functionality.
    - Use Case 1: [Name] Describe the first use case. Include the main flow and any exceptions.
    - Use Case 2: [Name] Repeat for second use case.
    - Use Case 3: [Name] Repeat for third use case.
    - Use Case 4: [Name] Repeat for third use case.
- Non-Functional Requirements: Describe system qualities like performance, usability, etc.
    - Usability: How easy should the system be to use?
    - Performance: Are there expectations around response time or efficiency?
    - Portability: Can it run on multiple platforms?
    - Reliability: Should the system be stable and consistent?
- External Interface Requirements: Only include if relevant. Otherwise, you can skip this section.
- User Interfaces: Describe major screens or interactions if needed.
- Software Interfaces: Mention if the software interacts with any other systems or libraries.